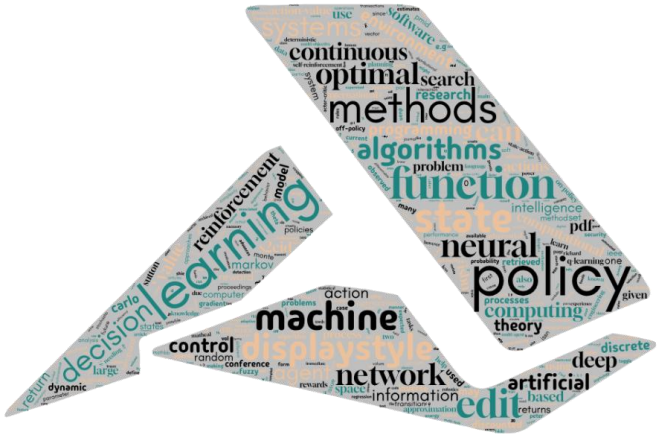


Reinforcement Learning Algorithms and Applications

Chapter 12. Proximal Policy Optimization for Acrobot

Acrobot Environment Overview

- **Acrobot-v1** is a classic *reinforcement learning* environment featuring a **double pendulum** system with underactuated control.
- The goal is to swing the *acrobot's end-effector* to a height at least **one link length** above the base.
- **Observation space** consists of *6 continuous values* representing joint angles, angular velocities, and system state. $s_t = [\cos\theta_1, \sin\theta_1, \cos\theta_2, \sin\theta_2, \dot{\theta}_1, \dot{\theta}_2]$
- **Action space** is *discrete* with **3 possible actions**: apply torque left, right, or no torque to the second joint.
- **Reward Definition**
  - $r_t = -1$ *per timestep*
  - Every step receives a reward of -1
  - Episode ends when the target height is reached or after 500 steps
  - Total *Reward* $= -($*number of steps*$)$
  - Faster completion $\rightarrow$ higher reward (closer to 0)
- Episodes are considered **successful** when the reward exceeds *-100*, typically achieved in fewer than **500 steps**.
- The environment provides a **challenging control problem** due to its underactuated nature and nonlinear dynamics.
- **Maximum episode length** is capped at *500 steps* to prevent infinite episodes.
- The reward structure encourages **efficient solutions** by penalizing longer episodes with negative rewards.

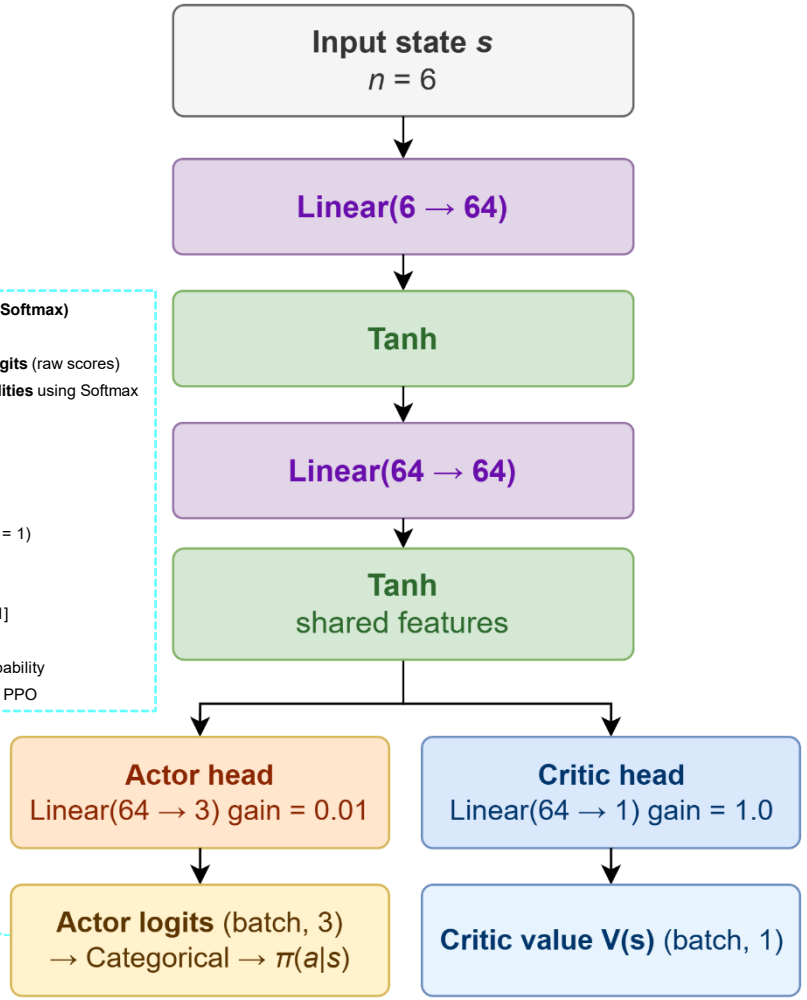
Proximal Policy Optimization (PPO)

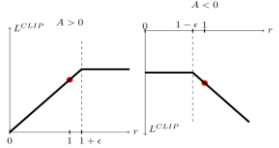
- **PPO** is a *policy gradient* method that addresses instability issues in traditional policy optimization algorithms.
- The core innovation is the **clipped surrogate objective** that prevents large policy updates during training.
- **Actor-critic architecture** combines policy learning with value function estimation for better sample efficiency.
- The *clipping mechanism* ensures policy updates stay within a **trust region** defined by the clip parameter.
- **Key advantages** include stable training, good sample efficiency, and robust performance across various tasks.
- PPO uses **Generalized Advantage Estimation (GAE)** with λ parameter for bias-variance tradeoff in advantage computation.
- **Multiple epochs** of minibatch updates are performed on collected experience for improved data utilization.
- The algorithm balances **exploration and exploitation** through entropy regularization in the objective function.

Generalized Advantage Estimation (GAE)

- **Start from Advantage Definition**
  - $A_t = Q(s_t, a_t) - V(s_t)$
- **Bellman Expansion of *Q***
  - $Q(s_t, a_t) = r_t + \gamma Q(s_{t+1}, a_{t+1})$
  - $A_t = r_t + \gamma Q(s_{t+1}, a_{t+1}) - V(s_t)$
- **Add & Subtract $\gamma V(s_{t+1})$**
  - $A_t = \underbrace{r_t + \gamma V(s_{t+1}) - V(s_t)}_{\delta_t \text{ (TD error)}} + \gamma\big(Q(s_{t+1}, a_{t+1}) - V(s_{t+1})\big)$
- **Substitute Definitions**
  - $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$
  - $A_{t+1} = Q(s_{t+1}, a_{t+1}) - V(s_{t+1})$
  - $A_t = \delta_t + \gamma A_{t+1}$
- **Expand Recursively**
  - $A_t = \delta_t + \gamma\delta_{t+1} + \gamma^2\delta_{t+2} + \cdots$
  - High variance (unstable)
- **Generalized Advantage Estimation (GAE)**
  - Introduce smoothing with λ:
  - $A_t = \delta_t + \gamma\lambda\delta_{t+1} + (\gamma\lambda)^2\delta_{t+2} + \cdots$
- **Recursive Form (Final)**
  - $A_t = \delta_t + \gamma\lambda A_{t+1}(1 - done_t)$
  - $Q(s_t, a_t) \approx R_t = A_t + V(s_t)$ (stable learning)
- **Key Insight** If $R_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \cdots$ (high variance)
  - δ_t: One-step TD error
  - A_t: Advantage
  - λ: Controls bias–variance tradeoff

- **From Logits to Probabilities (Softmax)**
  - **Idea**
    - Neural network outputs **logits** (raw scores)
    - Convert logits → **probabilities** using Softmax
  - **Softmax Formula**
    - $P(a_i \mid s) = \frac{e^{z_i}}{\sum_j e^{z_j}}$
    - z_i: logits
    - Output: probabilities (sum = 1)
  - **Example**
    - **Logits:** [2.0, 1.0, 0.1]
    - **Softmax:** [0.65, 0.24, 0.11]
  - **Interpretation**
    - Higher logit → higher probability
    - Used to sample actions in PPO





Assume:

$\epsilon = 0.2$, clip range $= [0.8, 1.2]$

A_t	ρ_t	clip(ρ_t)	$\rho_t A_t$	clip(ρ_t)A_t	min(·)	PPO Behavior
+1.0	1.5	1.2	1.5	1.2	1.2	Clip update
-1.0	0.3	0.8	-0.3	-0.8	-0.8	Clip update
-1.0	1.5	1.2	-1.5	-1.2	-1.5	Strongly penalize
+1.0	1.1	1.1	1.1	1.1	1.1	Normal learning

PPO Training Formulation

- **Advantage Normalization**
  - $\hat{A}_t = \frac{A_t - \mu(A)}{\sigma(A) + \epsilon}$, where ϵ is a very small constant
- **Importance Sampling Ratio**
  - $\rho_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$
  - $\log(\rho_t(\theta)) = \log \pi_\theta(a_t \mid s_t) - \log \pi_{\theta_{old}}(a_t \mid s_t)$
  - $\rho_t(\theta) = \exp\left(\log \pi_\theta(a_t \mid s_t) - \log \pi_{\theta_{old}}(a_t \mid s_t)\right)$
- **Clipped Policy Objective**
  - $L^{CLIP}(\theta) = \mathbb{E}[\min(\rho_t(\theta)A_t,\ \text{clip}(\rho_t(\theta), 1-\varepsilon, 1+\varepsilon)A_t)]$
- **Value Function Loss**
  - $L^{VF} = \mathbb{E}\left[\left(R_t - V(s_t)\right)^2\right]$
- **Entropy Loss**
  - $H(\pi) = -\sum_a \pi(a \mid s) \log \pi(a \mid s)$
  - $L^{ENT} = \mathbb{E}\left[H\left(\pi(\cdot \mid s_t)\right)\right]$
- **Final Objective**
  - $L(\theta) = -L^{CLIP} + c_1 L^{VF} - c_2 L^{ENT}$
- **Key Idea**
  - A_t: How much better an action is than average
  - Clipping: prevents large policy updates
  - Entropy: encourages exploration
  - Multi-epoch updates improve sample efficiency

Algorithm 1 PPO, Actor-Critic Style

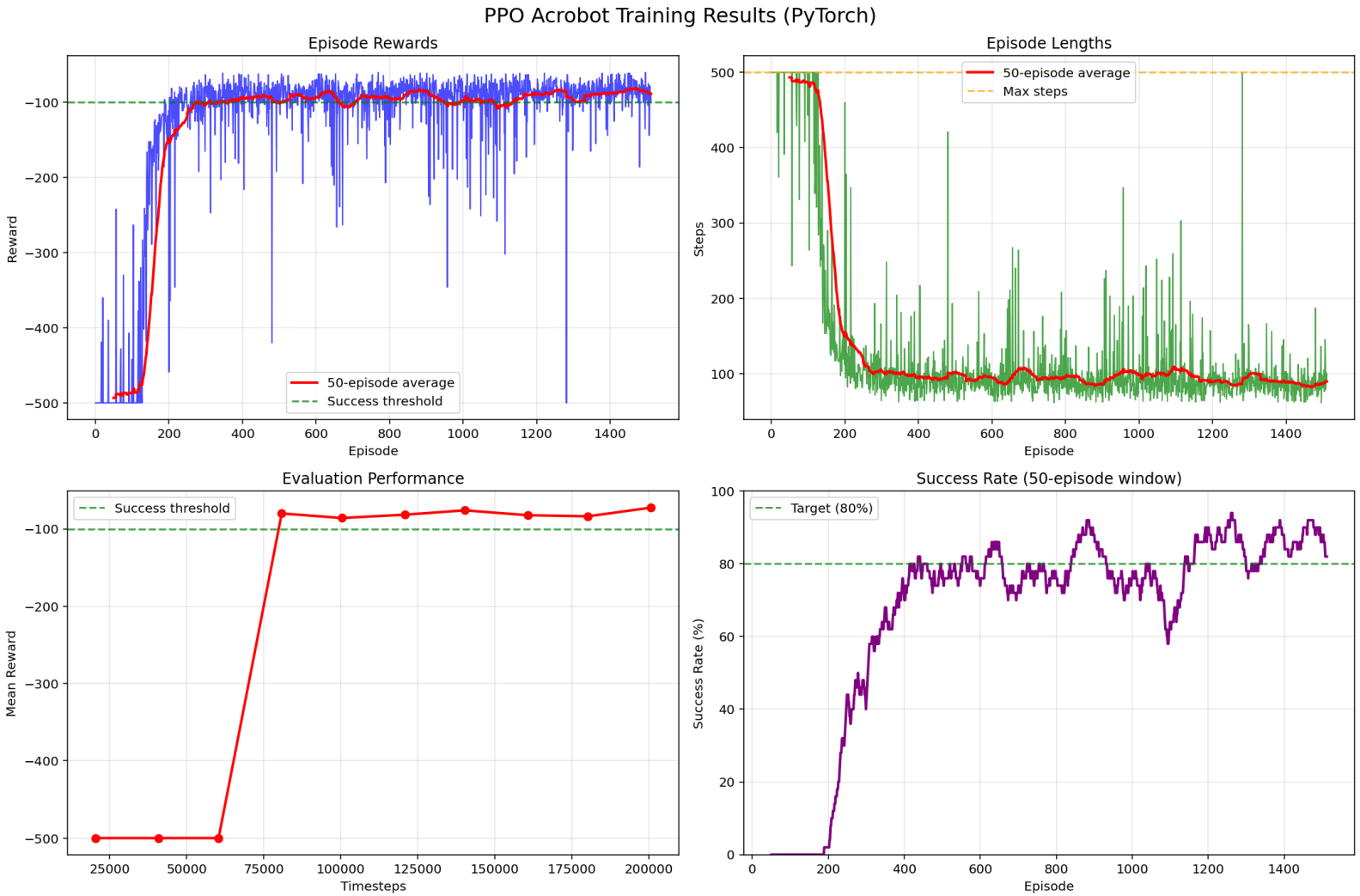
```
for iteration=1, 2, ... do          For every rollout (every rollout has T timesteps)
    for actor=1, 2, ..., N do       Here we only have one actor
        Run policy π_θold in environment for T timesteps      Run policy
        Compute advantage estimates Â_1, ..., Â_T             Compute advantages
    end for
    Optimize surrogate L wrt θ, with K epochs and minibatch size M ≤ NT
    θ_old ← θ   Update network      Train the actor and critic model together
end for
```

https://arxiv.org/pdf/1707.06347

- **Entropy Example (for PPO)**
  - **Policy Output (at state s_t)**
    - Assume the policy outputs probabilities:
    - $\pi(a \mid s) = [0.7,\ 0.2,\ 0.1]$
  - **Step 1: Entropy Formula**
    - $H(\pi) = -\sum_a \pi(a \mid s) \log \pi(a \mid s)$
  - **Step 2: Compute Each Term**
    - $H(\pi) = -[\,0.7\log(0.7) + 0.2\log(0.2) + 0.1\log(0.1)\,]$
    - $\log(0.7) \approx -0.357,\ \log(0.2) \approx -1.609,\ \log(0.1) \approx -2.303$
    - $H(\pi) = -[\,0.7(-0.357) + 0.2(-1.609) + 0.1(-2.303)\,] \approx 0.80$

Policy	Entropy	Meaning
[0.33, 0.33, 0.33]	≈ 1.10	Very random
[0.7, 0.2, 0.1]	≈ 0.80	Moderate
[0.99, 0.01, 0.00]	≈ 0.06	Almost deterministic

Training Result



Reference

- M. Lapan, *Deep Reinforcement Learning Hands-On*, 2nd ed. Birmingham, UK: Packt Publishing, 2020.
- A. Zai and B. Brown, *Deep Reinforcement Learning in Action*. Shelter Island, NY: Manning Publications, 2020.
- S. Ravichandiran, *Deep Reinforcement Learning with Python*, 2nd ed. Birmingham, UK: Packt Publishing, 2020.